

Introduction

The motivation for this project stemmed from the growing interest in real-time monitoring of Ethereum's mempool, particularly in the context of high-frequency blockchain interactions. Initially, the project was conceived with a focus on detecting and potentially front-running transactions, a use case commonly associated with arbitrage opportunities in decentralized finance. Front-running leverages mempool visibility to execute trades ahead of other users for financial gain. However, as we delved deeper into this idea, it became evident that front-running operates in an ethically grey area, raising concerns about fairness, trust, and the broader implications for decentralized ecosystems.

Recognizing these ethical challenges, we shifted our focus toward researching and developing tools that could provide value to a wider audience while avoiding ethically questionable practices. This pivot led us to explore the broader implications of sourcing Ethereum blockchain data, comparing the performance and costs of monitoring transaction activity using two approaches: a self-hosted Ethereum full node and a third-party provider like QuickNode. By benchmarking these two solutions, we aim to offer insights into how blockchain developers, researchers, and infrastructure providers can effectively and ethically leverage mempool data for a variety of applications.

The findings from this project provide valuable knowledge for a broader community, from developers working on real-time analytics and blockchain security to organizations optimizing infrastructure for decentralized systems. By focusing on practical and ethically sound use cases, this research aims to contribute positively to the growing ecosystem of blockchain-based technologies.

Design documentation for `full_node_monitor.cpp`

Overview

The `full_node_monitor_mempool.cpp` program is designed to monitor the Ethereum mempool using a full node and track transactions sent to a specific target address (the Uniswap Universal Router Address). By leveraging a locally hosted Geth node, the system ensures full control over the blockchain data and minimizes reliance on third-party services.

This document describes the design of the system, how it works, and how the understanding of Ethereum full node operations is utilized in the implementation.

System Design

1. Architecture Overview

The system interacts directly with a Geth full node via its JSON-RPC API. It performs the following core operations:

1. **Querying the Mempool**
Uses the `txpool_content` method to fetch details of all pending transactions from the Ethereum mempool.
2. **Parsing the Transactions**
Extracts transaction details such as the to address and the transaction hash.
3. **Filtering for Target Transactions**
Compares each transaction's to address with the specified target address to identify matches.
4. **Logging and Benchmarking**
Logs details of matching transactions and calculates the time taken to find a specified number of matches.

2. Key Components

Ethereum Full Node (Geth)

The full node is the backbone of the system. It operates as follows:

- **Data Synchronization:** Downloads and validates blocks to maintain a complete and up-to-date state of the Ethereum blockchain.
- **Mempool Management:** Maintains a pool of pending transactions awaiting inclusion in a block.
- **JSON-RPC API:** Exposes a programmable interface for querying blockchain data and interacting with the node.

Code Modules

1. **HTTP Interface (`get_txpool_content`)**
 - Sends JSON-RPC requests to the node.
 - Fetches the full mempool content using the `txpool_content` method.
 - Handles HTTP responses and errors gracefully.
2. **Transaction Filtering (`find_and_print_uniswap_transaction`)**
 - Parses the mempool JSON response.
 - Iterates through pending transactions to identify matches with the target address.
 - Logs matching transaction hashes and updates a counter.
3. **Benchmarking Logic (`main`)**
 - Tracks

the time taken to find a specified number of matching transactions using `std::chrono`.

- Implements a loop to repeatedly query the mempool until the target count is reached.

System Workflow

1. Initialization

- The program initializes by defining the target transaction count (target) and configuring the Ethereum node URL (ethereum_node_url).
- It connects to the Ethereum full node's JSON-RPC API endpoint for data retrieval.

2. Monitoring Mempool

- A JSON-RPC request is sent to the full node using the txpool_content method. This fetches all pending transactions grouped by the sender's address and nonce.

3. Parsing and Filtering

- The mempool data is parsed using JsonCpp.
- The program iterates through each transaction, checking if the to field matches the target address (Uniswap Universal Router).
- Each match is logged with its transaction hash.

4. Benchmarking

- The program calculates the elapsed time from the start of monitoring to the detection of the target number of transactions.
- The results, including total time taken and transaction hashes, are printed to the console.

Leveraging Full Node Operations

1. Mempool Insights

- The program leverages the txpool_content API, which provides granular access to pending transactions. This is crucial for understanding real-time activity in the Ethereum network.

2. Full Data Control

- Unlike third-party providers, running a full node ensures:

- **Unrestricted Access:** No reliance on service limits or blacklisting of specific contracts.
- **Real-time Data:** Immediate access to new transactions in the mempool without propagation delays.

3. Efficient Filtering

- By accessing raw mempool data, the program avoids unnecessary overhead associated with pre-filtered or delayed data from external services.
- This approach ensures that the program can benchmark transaction matching performance accurately.

Design Considerations

1. Scalability

- The program processes all transactions in the mempool sequentially. This is efficient for small to medium-sized mempools but may require optimization for high-traffic scenarios.

2. Error Handling

- Robust error handling ensures that network or JSON parsing errors do not crash the system. For example:
 - Invalid or empty responses are retried after a delay.
 - JSON parsing failures log errors and skip problematic entries.

3. Flexibility

- The target transaction count and polling interval can be easily adjusted for different benchmarking scenarios.
- The system can be extended to monitor multiple addresses or additional transaction attributes (e.g., gas price, sender address).

Design Documentation for quicknode_mempool_monitor.cpp

Overview

The `quicknode_mempool_monitor.cpp` project is designed to monitor Ethereum's mempool in two modes—serial and parallel—using the QuickNode third-party API. It detects transactions sent to the Uniswap Universal Router Address and benchmarks the time required to identify a specified number of such transactions in each mode.

This design document outlines the system architecture, components, workflow, and insights into how the code leverages QuickNode's WebSocket API for real-time transaction monitoring.

System Design

1. Architecture Overview

The system is built around QuickNode's WebSocket API, which allows for real-time subscription to pending Ethereum transactions. The program implements two monitoring modes:

1. **Serial Monitoring:** Processes each transaction hash sequentially.
2. **Parallel Monitoring:** Uses a thread pool to process transaction hashes concurrently.

Both modes aim to measure and compare the performance of QuickNode's data streaming capabilities in different operational paradigms.

2. Key Components

a. QuickNode WebSocket API

- Provides a new `PendingTransactions` subscription to stream real-time Ethereum mempool data.
- Allows retrieval of detailed transaction information using the `eth_getTransactionByHash` JSON-RPC method over HTTPS.

b. Monitoring Modes

1. **Serial Mode:**
 - Transaction hashes are processed one at a time.
 - Transaction details are fetched sequentially via HTTP requests.

2. Parallel Mode:

- Uses a thread pool (Boost Asio) with 15 threads to fetch and process transaction details concurrently.
- Ensures faster transaction matching by parallelizing HTTP requests and JSON parsing.

c. Benchmarking Logic

- Uses `std::chrono` to measure the elapsed time for each mode to process a user-defined number of matching transactions.

3. Functional Components

1. WebSocket Connection Initialization

- Creates and manages a WebSocket connection to the QuickNode API.
- Subscribes to pending transactions using the following message:

```
{"jsonrpc":"2.0","id":1,"method":"eth_subscribe","params":["newPendingTransactions"]}
```

2. Transaction Processing

- **Transaction Hash Extraction:**
Parses incoming WebSocket messages to extract transaction hashes.
- **Transaction Detail Retrieval:**
Fetches detailed transaction information using `eth_getTransactionByHash` over HTTPS.
- **Transaction Matching:**
Compares the `to` field of each transaction with the target address (Uniswap Universal Router).

3. Error Handling and Reconnection

- Implements WebSocket error and close handlers to retry connections in case of failures.
- Gracefully handles JSON parsing errors and malformed responses.

System Workflow

1. Input Parameters

- **Ethereum Node URL (WebSocket):** QuickNode WebSocket endpoint for real-time data streaming.
- **Ethereum Node URL (HTTPS):** QuickNode endpoint for JSON-RPC requests.
- **Transaction Count:** Number of transactions to match and benchmark.

2. Execution Flow

a. Serial Mode

1. Establishes a WebSocket connection to QuickNode.
2. Streams pending transaction hashes.
3. Processes each hash sequentially:
 - Fetches transaction details using HTTPS.
 - Matches the to field with the target address.
4. Logs matching transactions and increments the match counter.
5. Stops when the specified transaction count is reached and calculates elapsed time.

b. Parallel Mode

1. Initializes a thread pool with 15 threads.
2. Establishes a WebSocket connection to QuickNode.
3. Streams pending transaction hashes.
4. For each hash:
 - Posts an HTTP request to a thread in the pool to fetch transaction details.
 - Matches the to field with the target address.
5. Logs matching transactions and increments the match counter.
6. Stops when the specified transaction count is reached and calculates elapsed time.

Key Design Considerations

1. Performance Optimization

- **Parallel Processing:** Utilizes Boost Asio's thread pool to accelerate transaction detail retrieval in parallel mode.
- **WebSocket Streaming:** Leverages WebSocket's low-latency communication for real-time transaction updates.

2. Reliability

- **Implements reconnection logic in case of WebSocket failures.**
- **Gracefully handles HTTP request errors and malformed JSON responses.**

3. Flexibility

- **Supports different transaction targets and QuickNode endpoints as input parameters.**
- **Modular design allows easy extension to monitor additional transaction attributes or addresses.**

Benchmarking Logic

Serial Monitoring

- **Measures the time taken to process and match transactions sequentially.**
- **Suitable for scenarios with low transaction volume or minimal latency requirements.**

Parallel Monitoring

- **Benchmarks the time taken to process transactions concurrently using a thread pool.**
- **Suitable for high-throughput scenarios where processing speed is critical.**

Measurement documentation**Quicknode serial (ns)**

	1	10	20	40	60	80	100
Trial 1	1.00E+10	5.75E+11	7.59E+11	1.10E+12	2.48E+12	3.97E+12	3.35E+12
Trial 2	1.66E+10	3.02E+11	6.87E+11	7.83E+11	2.33E+12	3.90E+12	3.02E+12
Trial 3	8.19E+10	1.54E+11	4.89E+11	1.15E+12	2.40E+12	4.0E+12	3.00E+12
Average time	6.62E+10	3.44E+11	6.45E+11	1.01E+12	2.40E+12	3.96E+12	3.12E+12

Quicknode parallel (ns)

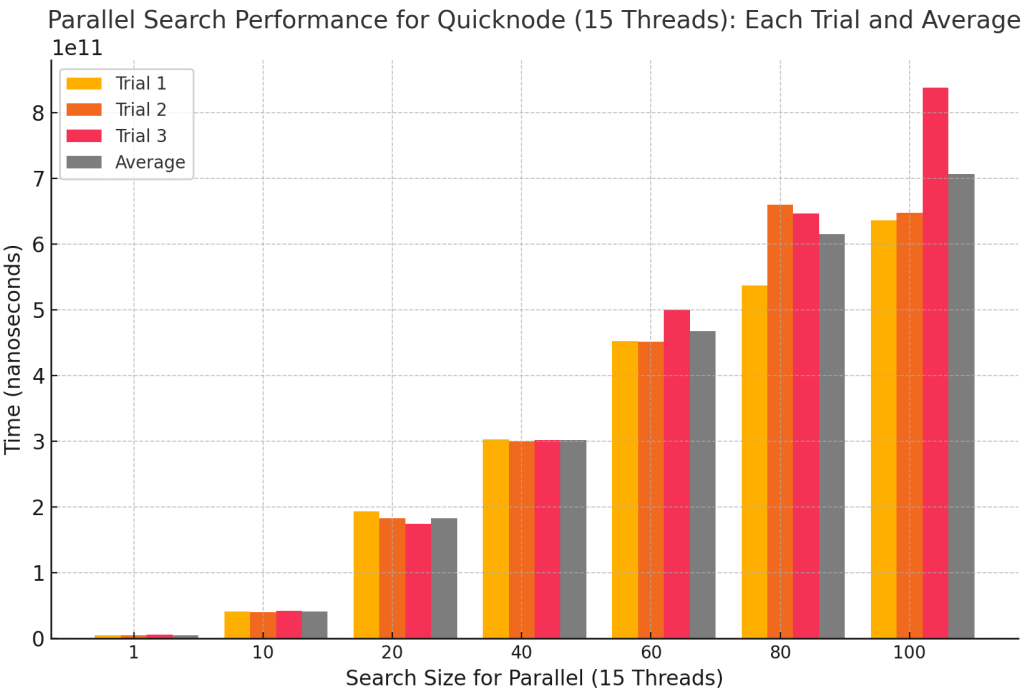
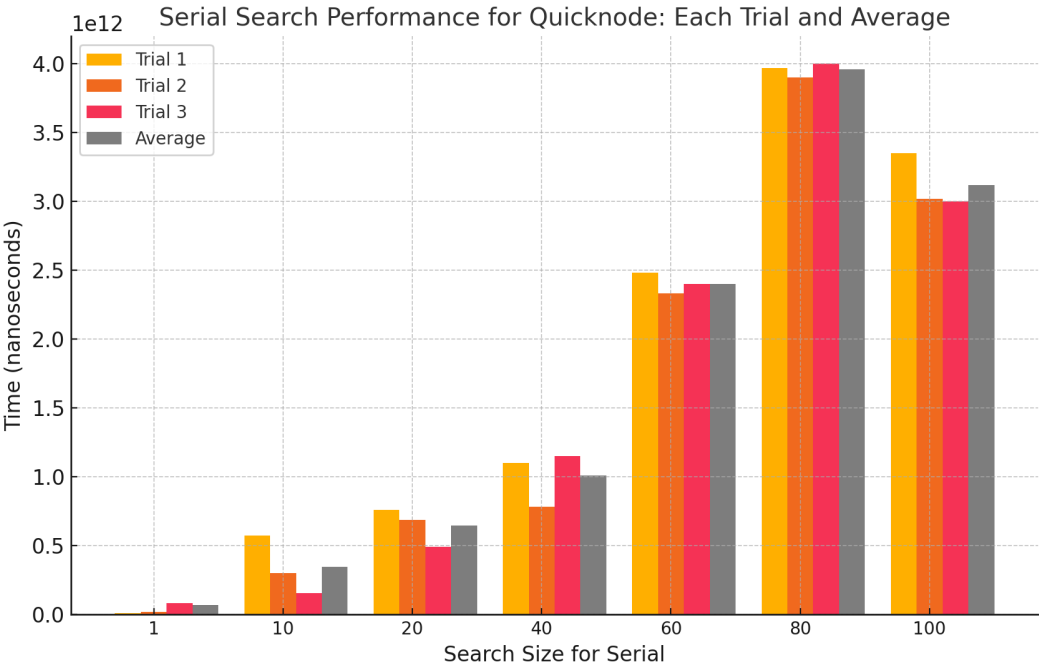
	1	10	20	40	60	80	100
Trial 1	4.87E+09	4.11E+10	1.93E+11	3.03E+11	4.52E+11	5.37E+11	6.36E+11
Trial 2	4.77E+09	3.99E+10	1.83E+11	3.00E+11	4.51E+11	6.60E+11	6.48E+11
Trial 3	5.81E+09	4.16E+10	1.74E+11	3.02E+11	5.00E+11	6.47E+11	8.38E+11
Average time	5.15E+09	4.09E+10	1.83E+11	3.02E+11	4.68E+11	6.15E+11	7.07E+11

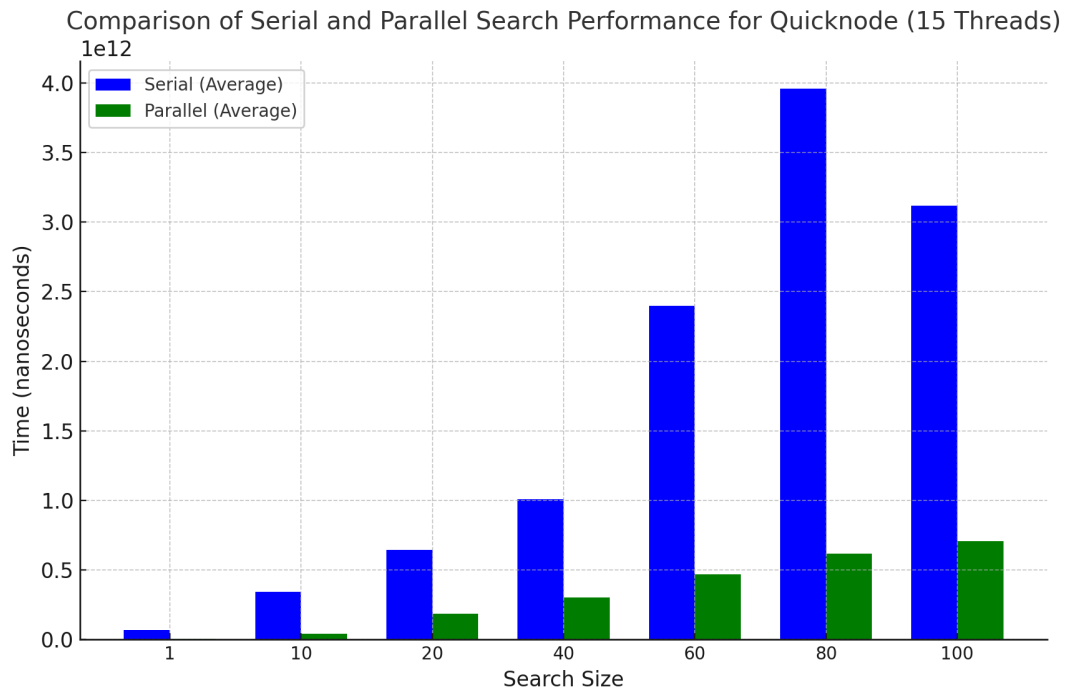
Full node serial (nanoseconds)

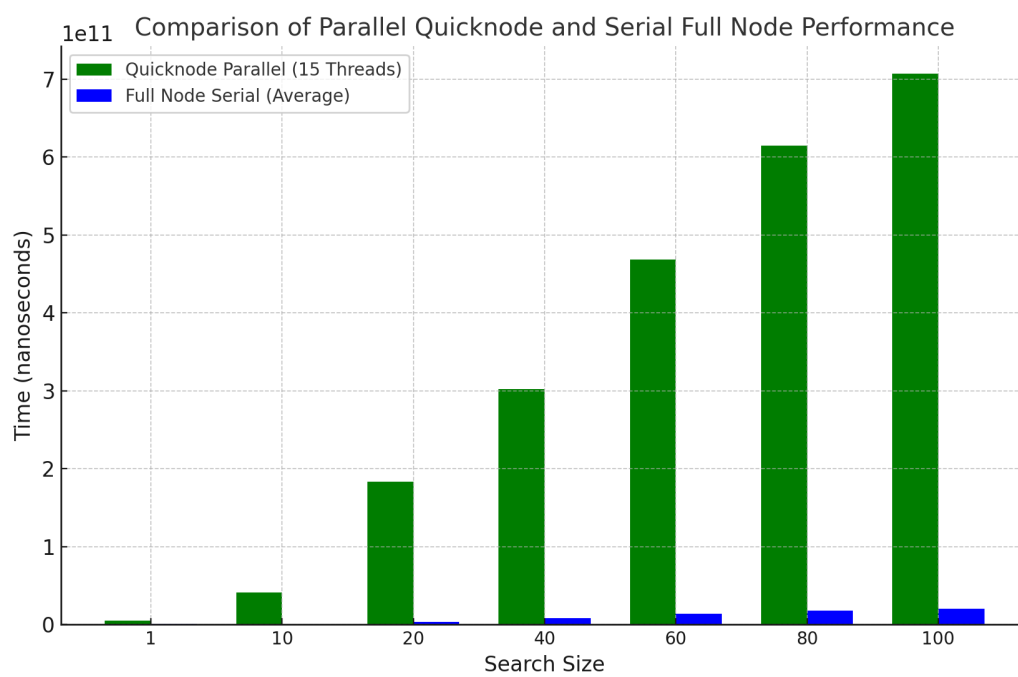
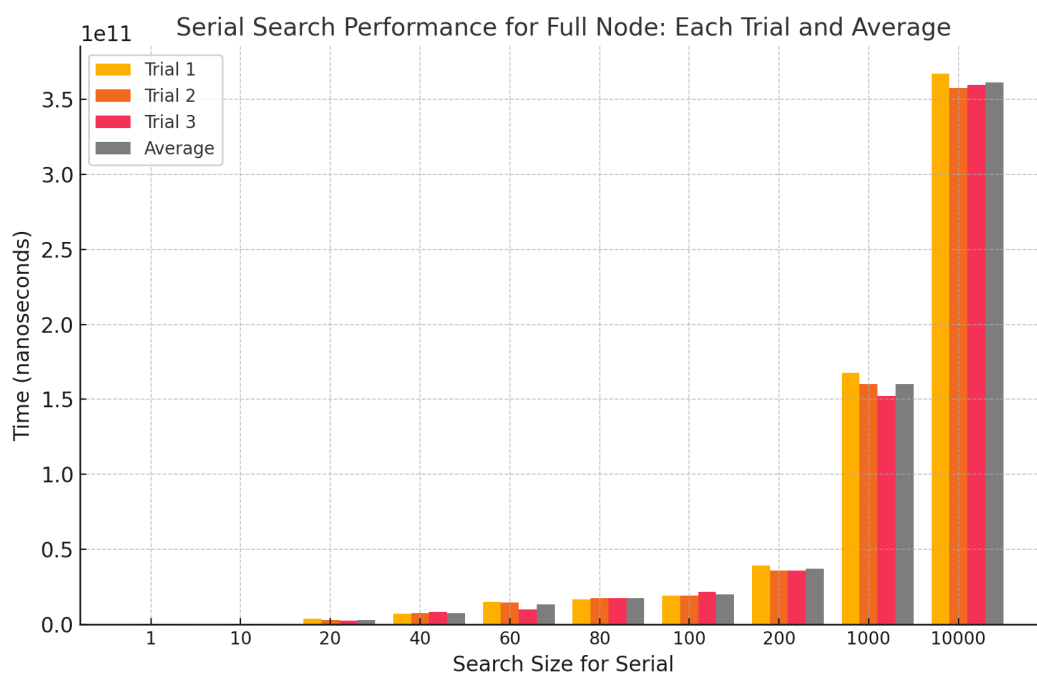
	1	10	20	40	60	80	100	200	1000	10,000
Trial 1	4.35E+08	5.34E+08	3.93E+09	7.27E+09	1.51E+10	1.68E+10	1.92E+10	3.91E+10	1.68E+11	3.67E+11
Trial 2	3.78E+08	3.38E+08	2.86E+09	7.53E+09	1.46E+10	1.77E+10	1.92E+10	3.60E+10	1.60E+11	3.57E+11
Trial 3	4.35E+08	3.32E+08	2.59E+09	8.29E+09	1.09E+10	1.77E+10	2.17E+10	3.61E+10	1.52E+11	3.60E+11
Average	4.16E+08	4.01E+08	3.13E+09	7.69E+09	1.33E+10	1.74E+10	2.01E+10	3.71E+10	1.60E+11	3.61E+11

time										
------	--	--	--	--	--	--	--	--	--	--

Charts







Discussion of measurements

The benchmarking results clearly highlight the trade-offs between QuickNode's API-based approach and a locally hosted full node in terms of performance and cost. In the QuickNode serial implementation, the average time to process 100 transactions was approximately 3.12×10^{12} nanoseconds (~52 minutes), while the full node completed the same task in $\sim 2.01 \times 10^{10}$ nanoseconds (~2 seconds). This means the QuickNode serial implementation, on average, is roughly 155x slower than FullNode. The primary bottleneck for QuickNode in serial mode is the sequential handling of HTTP requests, which introduces significant network latency and delays associated with parsing JSON responses.

Parallelizing the QuickNode implementation provides a notable speedup. For 100 transactions, the QuickNode parallel implementation took 7.07×10^{11} nanoseconds (~11.8 minutes), representing a 4.4x speedup compared to its serial counterpart. This improvement is achieved by utilizing a thread pool to concurrently process transaction hashes, reducing the impact of network delays. However, even with this optimization, the parallel implementation is still ~ 35 x slower than the full node. The gap underscores the advantage of direct access to the mempool in a full node, which eliminates the need for network-dependent API calls.

The cost comparison between the two approaches is also striking. A full node costs approximately \$600 per month to operate on a cloud platform like AWS, considering the compute, storage, and bandwidth requirements. QuickNode, on the other hand, costs \$30 per month for API access, making it significantly cheaper. When evaluating performance per dollar, the full node delivers exceptional value for high-throughput and latency-sensitive applications, such as real-time arbitrage bots or front-running detection, where its efficiency justifies the higher operational costs.

For practical use cases, QuickNode is ideal for projects with lower transaction volumes or early-stage prototyping where cost is a primary consideration. Its low setup cost and ease of use make it an excellent choice for developers exploring blockchain data without the need for ultra-low latency. Conversely, the full node is better suited for high-frequency monitoring or applications that require complete control over blockchain data, such as security-sensitive systems or infrastructure for decentralized finance. Organizations prioritizing data privacy and performance will find the full node indispensable.

In conclusion, the choice between QuickNode and a full node depends on the specific requirements of the use case. QuickNode offers an affordable and accessible solution for small-scale or non-critical applications, while full nodes provide unmatched performance and reliability for demanding, real-time blockchain operations. This benchmarking study provides a clear understanding of the trade-offs, helping users make informed decisions based on their operational priorities.

Practical Insights

The benchmarking results emphasize the trade-offs between QuickNode's API-based approach and a locally hosted full node in terms of performance, cost, and reliability. In the QuickNode serial implementation, the average time to process 100 transactions was approximately 3.12×10^{12} nanoseconds (~52 minutes), compared to $\sim 2.01 \times 10^{10}$ nanoseconds (~20 seconds) for the full node. This translates to the QuickNode serial implementation being approximately 155× slower than the full node. The delay is primarily caused by the sequential HTTP requests, where each network call introduces significant latency.

Parallelizing the QuickNode implementation mitigates some of these delays, reducing the time to 7.07×10^{11} nanoseconds (~11.8 minutes) for 100 transactions, resulting in a 4.4× speedup compared to the serial version. By utilizing a thread pool, the parallel implementation processes multiple transactions concurrently, effectively reducing network latency's overall impact. However, even with parallelism, QuickNode is still ~35× slower than the full node for 100 transactions. This highlights the fundamental advantage of the full node's direct access to the mempool, which bypasses network latency and reliance on remote API calls.

The cost analysis provides additional perspective. A full node costs approximately \$600 per month to operate, factoring in compute, storage, and bandwidth requirements, while QuickNode costs just \$30 per month. Despite its higher cost, the full node offers unparalleled performance and independence from third-party providers, making it suitable for latency-sensitive and high-throughput applications. QuickNode, by contrast, is more affordable and easier to set up, making it a practical choice for projects where performance is not the primary concern.

However, the full node's performance benefits come with significant operational challenges. A full node is your sole source of information, and in the event of a blockchain reorganization (re-org), where the chain is momentarily rewritten, the full node may start resending old transactions, leaving you temporarily without accurate or up-to-date data. This vulnerability underscores the reliance on a single point of data access, which can be problematic in dynamic or high-stakes environments. QuickNode, as a third-party provider, typically has redundancy measures in place to mitigate such risks, offering more consistent uptime and data availability during events like re-orgs.

Practically, QuickNode is well-suited for cost-sensitive or lower-priority use cases, such as prototyping, testing, or small-scale transaction monitoring. It provides an affordable entry point for developers looking to interact with Ethereum without investing in the infrastructure required for a full node. On the other hand, a full node is essential for high-frequency trading systems, security-critical applications, or projects that demand full control over the data pipeline, provided the risks of relying on a single data source are understood and mitigated.

In conclusion, while the full node excels in performance and independence, its operational complexity and risks in scenarios like re-orgs need to be carefully considered. QuickNode, although slower, offers affordability, simplicity, and higher resilience for less critical or cost-sensitive applications. This comparison highlights the importance of aligning the choice of infrastructure with the specific needs and constraints of your use case.

[illegible]

```

healedBytecodeBytes: 0,
healedTrienodeBytes: 0,
healedTrienodes: 0,
healingBytecode: 0,
healingTrienodes: 0,
highestBlock: 21329001,
startingBlock: 21329361,
syncedAccountBytes: 0,
syncedAccounts: 0,
syncedBytecodeBytes: 0,
syncedBytecodes: 0,
syncedStorage: 0,
syncedStorageBytes: 0,
txIndexFinishedBlocks: 2350000,
txIndexRemainingBlocks: 0
}
{
eth.syncing
currentBlock: 21329553,
healedBytecodeBytes: 0,
healedBytecodes: 0,
healedTrienodeBytes: 0,
healedTrienodes: 0,
healingBytecode: 0,
healingTrienodes: 0,
highestBlock: 21329001,
startingBlock: 21329361,
syncedAccountBytes: 0,
syncedAccounts: 0,
syncedBytecodeBytes: 0,
syncedBytecodes: 0,
syncedStorage: 0,
syncedStorageBytes: 0,
txIndexFinishedBlocks: 2350000,
txIndexRemainingBlocks: 0
}
{
eth.syncing
currentBlock: 21329619,
healedBytecodeBytes: 0,
healedBytecodes: 0,
healedTrienodeBytes: 0,
healedTrienodes: 0,
healingBytecode: 0,
healingTrienodes: 0,
highestBlock: 21329030,
startingBlock: 21329361,
syncedAccountBytes: 0,
syncedAccounts: 0,
syncedBytecodeBytes: 0,
syncedBytecodes: 0,
syncedStorage: 0,
syncedStorageBytes: 0,
txIndexFinishedBlocks: 2350000,
txIndexRemainingBlocks: 0
}
> exit
# ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$ df -h
Filesystem      Size      Used Avail Use% Mounted on
/dev/root        1.8T    1.2T    618G   67% /
tmpfs            1.8G    0      1.8G    0% /dev/shm
tmpfs            3.1G    932K    3.1G    1% /run
tmpfs            5.0M    0      5.0M    0% /run/lock
efivarfs         28K    3.6K    28K    3% /sys/firmware/efi/efivars
/dev/nvme0n1p6   881M    133M    687M   17% /boot
/dev/nvme0n1p5   105M    6.1M    99M    6% /boot/efi
                6.6G    12K    6.6G    1% /run/user/1000
# ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$

```

```
^C
ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$ g++ -std=c++17 -o pending_tx full_node_monitor_mempool.cpp -lboost_system -lpthread -lcurl -ljsoncpp
l -lcrypto
ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$ sudo ./pending_tx
Monitoring mempool for transactions to Uniswap Universal Router...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
No transaction found, retrying...
^C
ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$ geth attach
Welcome to the Geth JavaScript console!

instance: Geth/v1.14.11-stable-f3c696fa/linux-amd64/go1.23.1
at block: 21329430 (Wed Dec 04 2024 13:28:11 GMT+0000 (UTC))
datadir: /home/ubuntu/.ethereum
modules: admin:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> eth.syncing
{
  currentBlock: 21329437,
  healedBytecodeBytes: 0,
  healedBytecodes: 0,
  healedTrieNodeBytes: 0,
  healedTrieNodes: 0,
  healingBytecode: 0,
  healingTrieNodes: 0,
  highestBlock: 21329601,
  startingBlock: 21329361,
  syncedAccountBytes: 0,
  syncedAccounts: 0,
  syncedBytecodeBytes: 0,
  syncedBytecodes: 0,
  syncedStorage: 0,
  syncedStorageBytes: 0,
  txIndexFinishedBlocks: 2350000,
  txIndexRemainingBlocks: 0
}
> eth.syncing
{
  currentBlock: 21329447,
  healedBytecodeBytes: 0,
  healedBytecodes: 0,
  healedTrieNodeBytes: 0,
  healedTrieNodes: 0,
  healingBytecode: 0,
  healingTrieNodes: 0,
  highestBlock: 21329601,
  startingBlock: 21329361,
  syncedAccountBytes: 0,
  syncedAccounts: 0,
  syncedBytecodeBytes: 0,
  syncedBytecodes: 0,
  syncedStorage: 0,
  syncedStorageBytes: 0,
  txIndexFinishedBlocks: 2350000,
  txIndexRemainingBlocks: 0
}
> eth.syncing
```

Screenshots from fullnode tests

```

Transaction found to Uniswap Universal Router!
Hash: 0xb37a3a7528f4ce8d79efdc25d2ca459bd72dda4ba9d8f94dee8df750761
Transaction found to Uniswap Universal Router!
Hash: 0xb6e3cc7fe7886c4582a624744dea9ba7935dc04c9c85c5d8f83528648f55c10
Transaction found to Uniswap Universal Router!
Hash: 0x036da0e61ee280fe674cd0283166e9e9ce98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0xae21aab8eac8598b1911c8c8a4dfdc34609a16f3f78c29724b67a6ff5e585
Transaction found to Uniswap Universal Router!
Hash: 0x3f13df95127ef835f7605f355cba5e88bdaa2c4e78b2be616cf35ea3ea52ad
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd259363b1df2201db0b8853f4f8e666d7faf4bc7278
Transaction found to Uniswap Universal Router!
Hash: 0x5602e19088f86bdc2933c3404318dc023d873515537bf0282f66166a3926deb6
Transaction found to Uniswap Universal Router!
Hash: 0x480c413293d4b3f61cf167b0be3ac8c363851a90c996f274236b0638d7a3c572
Transaction found to Uniswap Universal Router!
Hash: 0xa50f54f6efb62f3beae52e2ff166f64bc9f1a692825f6bb4411155917facfb8f
Transaction found to Uniswap Universal Router!
Hash: 0x362e0b24349b2a149e8b194ea6d81c3d2177824a5b6e67830f8cea69cb1980
Transaction found to Uniswap Universal Router!
Hash: 0xa66d8ec6897a3fc7c866b4385af3899c2c714e279d1bc7734a3801798950a03d
Transaction found to Uniswap Universal Router!
Hash: 0x3cfff81be86226b90f85ecb1d4114ed09a254c7d2d43a8505ce46c91c58a2f
Transaction found to Uniswap Universal Router!
Hash: 0x17708c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139d439e4bc
Transaction found to Uniswap Universal Router!
Hash: 0xb37a3a7528f4ce8d79efdc25d2ca459bd72dda4ba9d8f94dee8df750761
Transaction found to Uniswap Universal Router!
Hash: 0x5257bf6e376666d4bedf82018f71aa0136539a336d7a9083fbd776b2983ede
Transaction found to Uniswap Universal Router!
Hash: 0xa6232a48a162fb7501f88c7dc7f43fec99deb09810b20e3382925842841cf
Transaction found to Uniswap Universal Router!
Hash: 0xb6e3cc7fe7886c4582a624744dea9ba7935dc04c9c85c5d8f83528648f55c10
Transaction found to Uniswap Universal Router!
Hash: 0x036da0e61ee280fe674cd0283166e9e9ce98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0x3f13df95127ef835f7605f355cba5e88bdaa2c4e78b2be616cf35ea3ea52ad
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd259363b1df2201db0b8853f4f8e666d7faf4bc7278
Transaction found to Uniswap Universal Router!
Hash: 0x5602e19088f86bdc2933c3404318dc023d873515537bf0282f66166a3926deb6
Transaction found to Uniswap Universal Router!
Hash: 0x480c413293d4b3f61cf167b0be3ac8c363851a90c996f274236b0638d7a3c572
Time taken to find 100 transactions: 19219517753 nanoseconds.
Transaction found, exiting.
uniswap-172-11-30-92:~$psql -u arbitrago

```

```

Transaction found to Uniswap Universal Router!
Hash: 0x17708c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139d439e4bc
Transaction found to Uniswap Universal Router!
Hash: 0xb37a3a7528f4ce8d79efdc25d2ca459bd72dda4ba9d8f94dee8df750761
Transaction found to Uniswap Universal Router!
Hash: 0x036da0e61ee280fe674cd0283166e9e9ce98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd259363b1df2201db0b8853f4f8e666d7faf4bc7278
Transaction found to Uniswap Universal Router!
Hash: 0x5602e19088f86bdc2933c3404318dc023d873515537bf0282f66166a3926deb6
Transaction found to Uniswap Universal Router!
Hash: 0x480c413293d4b3f61cf167b0be3ac8c363851a90c996f274236b0638d7a3c572
Transaction found to Uniswap Universal Router!
Hash: 0xa50f54f6efb62f3beae52e2ff166f64bc9f1a692825f6bb4411155917facfb8f
Transaction found to Uniswap Universal Router!
Hash: 0x362e0b24349b2a149e8b194ea6d81c3d2177824a5b6e67830f8cea69cb1980
Transaction found to Uniswap Universal Router!
Hash: 0xa66d8ec6897a3fc7c866b4385af3899c2c714e279d1bc7734a3801798950a03d
Transaction found to Uniswap Universal Router!
Hash: 0x3cfff81be86226b90f85ecb1d4114ed09a254c7d2d43a8505ce46c91c58a2f
Transaction found to Uniswap Universal Router!
Hash: 0x17708c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139d439e4bc
Transaction found to Uniswap Universal Router!
Hash: 0xb37a3a7528f4ce8d79efdc25d2ca459bd72dda4ba9d8f94dee8df750761
Transaction found to Uniswap Universal Router!
Hash: 0x036da0e61ee280fe674cd0283166e9e9ce98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd259363b1df2201db0b8853f4f8e666d7faf4bc7278
Transaction found to Uniswap Universal Router!
Hash: 0x5602e19088f86bdc2933c3404318dc023d873515537bf0282f66166a3926deb6
Transaction found to Uniswap Universal Router!
Hash: 0x480c413293d4b3f61cf167b0be3ac8c363851a90c996f274236b0638d7a3c572
Transaction found to Uniswap Universal Router!
Hash: 0xa50f54f6efb62f3beae52e2ff166f64bc9f1a692825f6bb4411155917facfb8f
Transaction found to Uniswap Universal Router!
Hash: 0x362e0b24349b2a149e8b194ea6d81c3d2177824a5b6e67830f8cea69cb1980
Transaction found to Uniswap Universal Router!
Hash: 0xa66d8ec6897a3fc7c866b4385af3899c2c714e279d1bc7734a3801798950a03d
Transaction found to Uniswap Universal Router!
Hash: 0x3cfff81be86226b90f85ecb1d4114ed09a254c7d2d43a8505ce46c91c58a2f
Time taken to find 80 transactions: 16636284519 nanoseconds.

```

```

Transaction found to Uniswap Universal Router!
Hash: 0x17780c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139dd39e4bc
Transaction found to Uniswap Universal Router!
Hash: 0x8b37a347528f4c6e8d79efdc252ca4459bd72dda4ba9d89f94deef8df750761
Transaction found to Uniswap Universal Router!
Hash: 0x036da8e61ee280fe674cd0203166e9e9cef98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0x68d0bde33ed66f3e0f351740a6705a8ea5eea9c5e22dafec37000b930d963aa2
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd25936eb41df2201db0b8853f4f8e666d7faf4bc7278
Transaction found to Uniswap Universal Router!
Hash: 0x5602e19088f86bdc2933c3404318dc023d873515537bfd282f66166a3926deb6
Transaction found to Uniswap Universal Router!
Hash: 0x362e0b24349b2a149e8b194ea6d8c1c3d2177824a5b6e67830f0cea69cbc1980
Transaction found to Uniswap Universal Router!
Hash: 0xa66d8ec6097a3fc7c866b4385af3899c2c714e279d1bc7734a3001798950a03d
Transaction found to Uniswap Universal Router!
Hash: 0x3cfff81be86226b90f856ccb1d4114edc09a254c7d2d43a8505ce46c91c58a2f
Transaction found to Uniswap Universal Router!
Hash: 0x17780c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139dd39e4bc
Transaction found to Uniswap Universal Router!
Hash: 0x8b37a347528f4c6e8d79efdc252ca4459bd72dda4ba9d89f94deef8df750761
Transaction found to Uniswap Universal Router!
Hash: 0x8fc5df3f1344aa2169bdda43b24b39e2ac50a549b3ff84a29af1d4e9b84e858
Transaction found to Uniswap Universal Router!
Hash: 0x036da8e61ee280fe674cd0203166e9e9cef98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0x68d0bde33ed66f3e0f351740a6705a8ea5eea9c5e22dafec37000b930d963aa2
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd25936eb41df2201db0b8853f4f8e666d7faf4bc7278
Time taken to find 60 transactions: 15119630964 nanoseconds.
Transaction found, exiting.
ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$ █

```

```

Transaction found to Uniswap Universal Router!
Hash: 0xa66d8ec6097a3fc7c866b4385af3899c2c714e279d1bc7734a3001798950a03d
Transaction found to Uniswap Universal Router!
Hash: 0x92ce72dd98732efb45ff0831c9c47cf46946b0d0185524472424d7b5ef5c1d6c
Transaction found to Uniswap Universal Router!
Hash: 0x3cfff81be86226b90f856ccb1d4114edc09a254c7d2d43a8505ce46c91c58a2f
Transaction found to Uniswap Universal Router!
Hash: 0x17780c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139dd39e4bc
Transaction found to Uniswap Universal Router!
Hash: 0x8b37a347528f4c6e8d79efdc252ca4459bd72dda4ba9d89f94deef8df750761
Transaction found to Uniswap Universal Router!
Hash: 0x036da8e61ee280fe674cd0203166e9e9cef98245e824bdba329227d5804aa182
No transaction found, retrying...
Transaction found to Uniswap Universal Router!
Hash: 0x8dd006342edda5b35809815bdc103db61f929b2cfa5e99c49b9ba18baef3d0f
Transaction found to Uniswap Universal Router!
Hash: 0x68d0bde33ed66f3e0f351740a6705a8ea5eea9c5e22dafec37000b930d963aa2
Transaction found to Uniswap Universal Router!
Hash: 0x5ce6e5616318403289dd25936eb41df2201db0b8853f4f8e666d7faf4bc7278
Transaction found to Uniswap Universal Router!
Hash: 0x5602e19088f86bdc2933c3404318dc023d873515537bfd282f66166a3926deb6
Transaction found to Uniswap Universal Router!
Hash: 0x362e0b24349b2a149e8b194ea6d8c1c3d2177824a5b6e67830f0cea69cbc1980
Transaction found to Uniswap Universal Router!
Hash: 0xa66d8ec6097a3fc7c866b4385af3899c2c714e279d1bc7734a3001798950a03d
Transaction found to Uniswap Universal Router!
Hash: 0x92ce72dd98732efb45ff0831c9c47cf46946b0d0185524472424d7b5ef5c1d6c
Transaction found to Uniswap Universal Router!
Hash: 0x3cfff81be86226b90f856ccb1d4114edc09a254c7d2d43a8505ce46c91c58a2f
Transaction found to Uniswap Universal Router!
Hash: 0x17780c7d556a1f30af882ed46fd4e4275d70bd9a834fb87a747df139dd39e4bc
Time taken to find 40 transactions: 7266894377 nanoseconds.
Transaction found, exiting.
ubuntu@ip-172-31-39-92:~/Speed_n_arbitrage$ █

```

Screenshots of quick node tests

```

0xd59ef29e5aab4aa4ba36b276589ae196a8cbf0a2c390b758242f8bb26e00c47f (93)
0x40d74fee72e4e6ca053bb8a5af103337629331522c5a456675d86984345441ee (94)
0x48b511334cf44d8d27cf477f2e4380545036f7156768b447a8e04c249292fb20 (95)
0x683cdb89fc40709c2355ea4d9ddac79bda4902beef9f0eab58dd48ff991954e3 (96)
0x67122b74c299a02efbd9773324a05a3fda3ea4fae3a43ca560d8ca1b54d553fd (97)
0x65ee7d910681b8f128487c64373b1ab1b42cc0b4a5f6f92e3940b7755bfff1876 (98)
0xec1aa962add70142162b7d7a93e5ae2207ed716fc3fef9b3fccb594c6563561e (99)
0x93e5e4eedeb4fc69ce6cbb409a7ad5705e030b838ee05020a1f771edb253d94b (100)
PARALLEL TIME: 647570349000

```

```
0x8484c446548dda9413a0c4e8507bc62807aa8862086b170d5b32ac33c5f12daa (93)
0xd020c01cc5b71cdd4c18e17493e91f96823260e7460a60cc5dce5fc46e83f1d2 (94)
0x41ab3e640b675591b96b2c172f64cd2452b039485759b8950941333b26e1d042 (95)
0x5b7801387e157bee6f5c2cdf3ee348feb70690e749a96682e2282f0b74945ede (96)
0x2c9acd2fdfb34149f9c8ebb2922d214b04e03a41b4fc500190a143d591e33f48 (97)
0x830b53650d894851ed179113cc43762737509db4e695c7fd3e5fd23560aac023 (98)
0x757c40aa3c0e7206663c89be71607299551e958829e746088484ab74c537f8de (99)
0xdf1e183add08b83a7b443b6e1d632351e6c6da8cafd31668402f71abe7c89fef (100)
PARALLEL TIME: 838191430000
```

SERIAL TIME: 3022392679000

N=80

```
0x2978f67c98a58db6e4404eb15973daa32a514f7f5a8c7a7f98e547a79913d27c (73)
0x1c0bea341b7bbbf76bd26b75dfd88a102e76b5ace7b90e1e94601645be29a0af (74)
0x64b896bf2418269e5de2c999590b34bac5c50987225d87e8349bdd832cb3e420 (75)
0xbaa89ecdc6c9b870263f57c2979177ee6a9544f46318bf72fe3fca201c3abcbb (76)
0xc30c74f3c5bc53d3d563e50a68f9bdbd18c73effcfe1b2533dec7673d1c33d9c (77)
0x639492cdfd7e6ecfed61aefa8e98b1ce1a17114e38aa3a77c1ccfe7d73af20d3 (78)
0x91ac0a5228bef50214ba24fbf86f0dadec3c01496791de056d1eb4455b1fd731 (79)
0x083990e05a605ed52679a7366abecd9ac9ae28e9aa83b9cce990cbe42ec97383 (80)
PARALLEL TIME: 537431510000
```

```
0x428e8121b1505ea38f3c1b20916de65f1bea61ec83fb2702bd2a4ee8ba17cd48 (73)
0xc7449f0b0583665f47fff9220007431897679b12f01644bcca8ca3483237f3ed (74)
0x21e8e70c9d42990e1b474ccb27a86ffd835f5caa49e71f124322e1447e0f1615 (75)
0x52d9e646b7a7204420fdbf3efa4d7f80c14aab1b4bf9a0e3ba445f30c325ec2a (76)
0x8210b236a3424c0142c0b7b32fd376ee18feadadc9cfee07a6e37a93094d6a61 (77)
0x1d732cba8a73e27b2f896e466c9f544177d701cb79cb97019ab66febda968e0a (78)
0xc7eb59994511082998e87be125f4ae5cfc69b4e00a0854f09d4aa892908d354f (79)
0x572714df7e4f7aa026ffdd9c3dee758e94fb576445420bb38bbfa337cb350d76 (80)
PARALLEL TIME: 660375167000
```

```
0x1f72012de0ba3ba134cbabec8f75663458e212218fb4499d61d7a986b73b4c1c (73)
0xf20b0c1211423d2f97cccc37ec1c3d3624a7149293cda02e7bab6148dbe4ad04 (74)
0x756dd8da0ae0f8ea6c9789d3e9d83a0899ea60d6c75dab764b49cbfb3e4620ce (75)
0xfea5cb052f5cb5650752847ccbc09c5919f34ac6a3ba26acd658c6a7e6e65fb8 (76)
0xc5f2bc944487a451761d435dc686ebfa5238aeb41603bbabb152ee112340fdf1 (77)
0xce92fd7f16eacbc5a4a8c0a08ca770e7bcb7bc013a568e212a8e47888982805a (78)
0xbaf4f49a9f66d9eec9ee8e92c5523ee170fe10178421d06a32b7e54db20e7ad6 (79)
0x7cda3170ef5f6ea5ab4602947b1c47f56f53a5bb633c04177c908acade5dd43e (80)
PARALLEL TIME: 647370192000
```

SERIAL TIME: 3972272191000

N=60

```
0xe14853609d7a9a773f17d51efc102a726fe0c888c3eccf0f9d5cb561f5874c21 (54)
0x810446c983612788360dd3581f5ff056820b311c6356f4222504095bb2d2c6b9 (55)
0x8f37077f6ddd0b87d0adb4cb46e3fdf70c6cb21c498d737bd1620e6eece94ce2 (56)
0xea7e0abb260933d2f4aded19f0cb32dd2900ba4c7cdc10bd9cc9d6f240781c6e (57)
0x000aa9cc382a7ec1bd7c3a242bf93ebdcf1334aaa80a61603eec6b8f40772556 (58)
0xb3423225463e56733b1eec009570e42a73d4759b3fb667bbcd7fffd8cb371d7d (59)
0x9b7e434f7edc9d933b21efb5a6c5a1c2c6b03969f05d769f3457d2c16c561ac2 (60)
0x913bfc28670ba097a3413c9d6d2296262caf62f517b76d0d3d8ccb1ecbdac14a (61)
PARALLEL TIME: 451830219000
```

SERIAL TIME: 2478344181000

N=40

```
0x9ad059c7238ddef fba7f7b35c91f571da63037f76c89606f70232e135cdb663c (33)
0x00e34aa975438d5f91e40ad7aa364c559525a0c812194671eba86fada383e4bb (34)
0x6bd67607fb01a43c1e869f0f72be1ed35a4e1ef8f0300556a51c9a4df75d0525 (35)
0x653a4844765a61202d10c544c513409b8a8df13743a1a83a99b8974ab4e40f83 (36)
0x74d410ec44c987c7740d10edc29221079094fdad3ad4bc0de603ad294019b1fb (37)
0x1fbed67fa19f0743f3e4a5ee8ed4533a02dcb3ce0ce6a0255c9df0c60414ce12 (38)
0xe15291b9a9e2af581d604517e4ff02c83865ef1eee0931e4f1ee50dc719dc834 (39)
0x9a461110c6a7958f82634ecd150f0e9d018c83437683d3015e669c8cbb6305b2 (40)
PARALLEL TIME: 303043046000
```

SERIAL TIME: 1094213953000

SERIAL TIME: 783333796000

SERIAL TIME: 1149675414000

N=20

```
0x68b17e12c5ce84c829fa33cfe256649cdb7f4c33b222a2ca3200f5f419151195 (13)
0x46dcd98083c0191faf5b4ca00b558210f705e350f8ab4b55948157592f9dfd3a (14)
0x8bf9cc0e832d0b0d29dfa52795d49761db38c5cc08ab19e4d83a65db05971323 (15)
0x9400496114006aa6e752d39ae5a8be234fde326f25ed04283a93f3078920d032 (16)
0xdf469b985b96086481057624e66b5f035e467299b368ae0c62fe463e56114404 (17)
0x3cfc334f40356215d9b616967a7e286e8b79e3cebc5c4cd0391f1c499d4a6b06 (18)
0x9a0d1f0a6c2d3b713332cb57aad8ce57dbfa54361805fd2cc9e3a7a4eb6e3b43 (19)
0x81117940e03e4dab8de381a6939e6e39753b969d560b77b440dff861025d8ed0 (20)
PARALLEL TIME: 183247841000
```

SERIAL TIME: 759053128000

SERIAL TIME: 686767608000

```
0xdafe5459aa6ff6e0add9886e04ac6d0e830e21a20497cc9e89ff8440d10c084b (17)
0x9b6f82423e5839cf1ded952d9a25f99bc925906135342e3caafa73930b04cc55 (18)
0x48b28395e3671aa1d3db86fab4300752b01162712ee9ccf532ef4f2855431456 (19)
0x7dba57fe932ce87cefe79c80a681764ed8c805e288f6f28026551cdae6d780e4 (20)
[2024-12-05 00:56:20] [info] asio async_read_at_least error: asio.ssl.s
[2024-12-05 00:56:20] [error] handle_read_frame error: websocketpp.trans
[2024-12-05 00:56:20] [info] asio async read at least error: asio.ssl.s
[2024-12-05 00:56:20] [error] handle_read_frame error: websocketpp.trans
SERIAL TIME: 489205656000
```

N=10

```
0x42a06865fbf011304455f685d5e6f7e50b2c2780c505d79585c8673616a0113d (3)
0x3e068d35734c91b86eea7fd96ab1d9eb32fa5675b5e60f68917389e653cb4be8 (4)
0x50f163316462e057dddf8c02fc53579d1b7263ece54b848fce72ad9702408c17 (5)
0xff13aea80738cf5682bb734749df1dee2f269ae638bef0b47065938298a09577 (6)
0x85a7cfa25cce3126d0e9ed07ff3eaf572b88b021645a777f8b479addb0ffc965 (7)
0x8ac85290e5cb33715fc17f6510888986a450d5d9b4b9708f7a142abfea5fc1d7 (8)
0xb496e9dac111315a11a19c10944f7edc428765335ebccc2afa2653b6ea13d869 (9)
0xb0ce79f732dd287d7796346652dc0be4cd529cc0ca064e8e1a1a4eb1d124d068 (10)
PARALLEL TIME: 41052825000
```

SERIAL TIME: 574708243000

```
0x141d7878d08a27e9f23ac5990824c714bac1a5fa33e2930807bf85e9937216bb (6)
0xbef0fa332851d07ada4336943967606f2a9c4995570865d153bbfa82d5016e15 (7)
0xf690cc7265119d4cad5b98db74ef5d33948fe69b8ddb0ed11a59a0cae65db1c0 (8)
0xaedf6324aa14d9d35e5237f3f57f569a9aaa82c930cbade32f107daf0908d4d3 (9)
0x18ad79350c780ae7acc423d2d4b9db616a378d0e662aa377f37ba01ded2a38da (10)
[2024-12-05 01:07:57] [info] asio async_read_at_least error: asio.ssl.st
[2024-12-05 01:07:57] [error] handle_read_frame error: websocketpp.trans
SERIAL TIME: 302251143000
```

SERIAL TIME: 154487850000

N=1

```
Subscribed to pending transactions
0x1616138c39727aa3e9da304b4a7148947ec7024acc177e1a1d8ecb24f56e89a6 (1)
PARALLEL TIME: 4874802000
```

SERIAL TIME: 100039093000

```
0xf0ba1b7ff48b6d9a9a9ffdae82f2a1161f7b12da85ab84ef95c1117fb20f7991 (1)
SERIAL TIME: 16611938000
```

Acknowledgements:

- **Blockchains Explained**
 - <https://www.youtube.com/watch?v=Jc0NwgqCtik&feature=youtu.be>
- **Inspiration for the project, and good guide on front-running.**
 - <https://medium.com/@ivanbogaty/ front-running-bancor-in-150-lines-of-python-with-ethereum-api-d5e2bfd0d798>
- **Guide on front-running**
 - <https://www.youtube.com/watch?v=uElOqz-Htos&feature=youtu.be>
- **Libraries Used**
 - boost, websocketpp, openssl@3, curl, jsoncpp
 - The C++ Standard Library